AI QA AGENT · READYTECH

Project Argus
An AI agent that tests tickets and learns each product

Argus reads a Jira ticket, plans the tests, runs them in a real browser, reports with screenshot evidence and leaves Playwright regression tests behind. Every ticket it tests makes it better at that product.

Sreeram Goalla · Product, Business Analysis & QA · one of 65 contributors across ReadyTech

Introduction

QA across a portfolio of SaaS products means re-learning each product's rules, roles and quirks for every ticket. Argus is an AI QA agent, built on Claude Code with Playwright browser automation and Jira and GitHub integrations, that carries that knowledge from one ticket to the next and applies the same rigorous method every time.



Tests real tickets

Reads the ticket, comments, linked docs and the code change, builds a test plan, and runs it in a real browser against a test environment.



Beyond the happy path

Permission matrices, exploratory edge cases, error monitoring and cross-product checks, all backed by screenshot evidence.



Leaves regression behind

Turns passing tickets into Playwright tests using the Page Object Model, and opens a pull request to the product's test repo.



Learns every product

Feature quirks, regression patterns and coverage flow back into a per-product knowledge base after every ticket.

My role

Argus was built by contributors across ReadyTech. I was one of its most active, **second of 65 by commits**, and used it every day on our student management product. I helped shape how it should earn trust, wrote the product knowledge it tests against, and ran it on real tickets.

PRODUCT MANAGER

Shaped how an AI agent earns trust

- Helped define the autonomy roadmap: per-product levels for removing human approval steps, each unlocked by measurable evidence.
- Backed safety rails that never go away: human review of every generated pull request, no testing in production, and human escalation for payments, authentication and data integrity.
- Framed the value for squads: faster regression, and consistent evidence on every ticket.
- Made Argus part of our squad's everyday QA, across grooming, testing and bug verification.

What I produced: Autonomy roadmap · Success criteria · Squad adoption

BUSINESS ANALYST

Wrote the product knowledge the agent tests against

- Wrote the Higher Education knowledge base: Curriculum Builder, self-enrolment, CSP clusters, SA-HELP and default enrolment outcomes.
- Captured what I learned from 114 tickets into feature rules, quirks, regression patterns and coverage.
- Documented business rules precisely enough for an agent to act on, such as enrolment patterns configured at both system and course level.
- Took every knowledge update through pull-request review, and addressed the review findings.

What I produced: Feature docs · Regression patterns · Coverage tracker

QUALITY & TESTING

Ran it on real tickets, every day

- Used Argus to groom, test and reproduce bugs on student management, student portal and curriculum tickets.
- Reviewed each run's report and evidence before it reached Jira, correcting or escalating wherever judgement was needed.
- Turned passing tickets into Playwright regression specs, and kept Higher Education coverage current.
- Kept the knowledge loop turning, so each ticket made the next run on that feature faster and sharper.

What I produced: Test reports · Playwright specs · Bug reproductions

284	114	24	5,000+
my commits, 2nd of 65 contributors	tickets captured in the knowledge base	products onboarded across ReadyTech	commits in the first 3 months

How it works

A small set of commands covers the whole QA lifecycle. The core loop runs once per ticket, and every pass feeds the knowledge base.

Onboard product → Groom ticket → **Run ticket** → Update knowledge ↺ Verify bug

Command	What it does	Output
Groom ticket	Analyses the ticket and code before testing: acceptance criteria, testability, permissions, edge cases and risk.	Test cases, plus questions for the developer
Run ticket	The full QA workflow: plan, execute, explore, report, automate and learn.	Jira report, Playwright PR, knowledge PR
Verify bug	Three reproduction attempts with variations, then a blast-radius check across roles and related screens.	Reproduced, cannot reproduce or partial, with evidence

Command	What it does	Output
Update knowledge	Records what a session learned: features, baselines, regression patterns and coverage.	Updated product knowledge
Onboard product	A guided session that discovers a product's stack, roles, environments and workflows.	A ready-to-test product knowledge base
Onboard yourself	Sets up a new team member's credentials and checks every integration.	A verified, working setup

Inside a ticket run: 11 steps

1. **Load context**
   Product config, knowledge base and environments.
2. **Read the ticket**
   Comments, linked issues, docs and the code diff.
3. **Pick the environment**
   Resolves where to test for this ticket.
4. **Build the test plan**
   Positive and negative cases, permissions, and cases derived from the diff.
5. **Execute**
   Structured tests with screenshot evidence.
6. **Explore & monitor**
   Edge cases, error tracking and cross-product checks.
7. **Human review**
   The full report is shown to a person for approval.
8. **Post to Jira**
   Evidence, comment, label and status.
9. **Clean up**
   Browser, screenshots and seeded data.
10. **Automate**
    Generates or updates Playwright specs, and opens a PR.
11. **Learn**
    Updates the knowledge base, and opens a PR.

Testing layers

Every run goes through the same layers, so even a small ticket gets more than a happy-path click-through.

Layer	What it checks
Structured	Each acceptance criterion with screenshot evidence; a baseline check that unchanged behaviour still holds; positive and negative paths; every permission against every relevant role; and what happens after a change (messages, audit trails, lists, the back button). The browser console is checked after every action.
Exploratory	Boundaries and odd inputs, data scoping by role, double-submits, invalid API payloads, related features, known regression patterns near the change, and records in unexpected states.
Monitoring	New exceptions, performance regressions and error-log patterns that appear during the test window.
Cross-product	Where a feature integrates with another product, checks the other side of the integration too.
Bug reproduction	Three attempts with variations, a blast-radius check across roles and screens, and a clear scope of who is affected.
Automation	Playwright specs using the Page Object Model, with a coverage-gap analysis and data seeding, delivered as a pull request.

A knowledge base that grows

Argus separates **how to test** from **what to test**. The method is shared by every product; the product knowledge grows with every ticket tested.

Agent level · how to test

SHARED BY EVERY PRODUCT

- Workflow commands
- Testing methodology
- Report formats
- Environment strategies
- Data seeding

Never contains product-specific details, so one improvement benefits every product.

Product level · what to test

GROWS WITH EVERY TICKET

- Overview & architecture
- Environments & roles
- Feature docs
- Behaviour baselines
- Regression patterns
- Related features
- Coverage
- Per-ticket history

By the tenth ticket on a product, the agent already knows its gotchas.

24 products onboarded across five business segments

9	9	2	2	2
Education & Work Pathways	Workforce Solutions	Government	Justice	Procurement

Autonomy roadmap

Argus earns the removal of human approval steps one product at a time, on evidence, and can lose it again. Each product's level lives in its configuration, so every promotion or demotion is a reviewed pull request.

Level	What changes	Entry criteria (per product)
L0	Today: a person approves every report.	Default
L1	Clean passes post automatically. Failures, escalations, payments and authentication always go to a person.	20+ runs in 90 days; zero confirmed false positives in 60 days; hard-intervention rate under 10%; full evidence coverage; no stale follow-ups
L2	Auto-planning on documented features, and automatic status transitions. Knowledge PRs are still reviewed by a person.	30 consecutive L1 runs with no overridden reports; zero false positives in 90 days; interventions under 5%
L3	Unattended scheduled runs.	Two quarters at L2; interventions under 5% across 50+ runs; organisation sign-off

Automatic demotion

Any confirmed false positive drops the product one level immediately. Any integrity incident sends it straight back to L0.

Never removed

Human review of every generated PR, no testing in production, and a person for anything touching payments, authentication or data integrity.

Measured weekly

A periodic audit produces the metrics every level is judged on, so the evidence is always current.

Summary

Project Argus, at a glance.

Capability	What it delivers
Ticket-driven QA	Grooming, testing and bug reproduction straight from Jira, with screenshot evidence posted back.
Layered testing	Structured, exploratory, monitoring and cross-product checks on every run.
Regression as a by-product	Passing tickets become Playwright specs, delivered as pull requests.
Knowledge that compounds	A shared method, plus per-product knowledge that grows with every ticket.
Portfolio scale	24 products across five business segments, and 65 contributors.
Earned autonomy	Human approval steps removed per product only on evidence, with automatic demotion and permanent safety rails.

🚀 **The takeaway:** Argus gives every squad a tester that never forgets a product's rules, applies the same method every time, and leaves regression coverage behind, while people stay in charge of the judgement calls.